

Advance Database Management System



By

Awais Ahmed

(2023-Arid-3627)

Sammar Abbas

(2023-Arid-3899)

Bachelor of Science in Artificial Intelligence (BSAI)

Barani Institute of Information Technology

PMAS Arid Agriculture University Rawalpindi

Spring 2026

Table of Contents

- Overview2
- Section 1 — Tables2
 - 1.1 customers.....2
 - 1.2 bills3
 - 1.3 consumption3
 - 1.4 payments.....4
 - 1.5 users4
- Section 2 — Relationships5
- Section 3 — Stored Procedures6
 - 3.1 Insert Procedures6
 - 3.2 Update & Delete Procedures6
- Section 4 — User-Defined Functions.....7
 - 4.1 Bill Status Functions.....7
 - 4.2 Customer & Consumption Functions8
- Section 5 — Summary.....8
- Section 5 — Triggers.....10
 - 5.1 Trigger Definitions10

Table of Figures

- Figure 1: customers — Table Definition.....4
- Figure 2: bills — Table Definition4
- Figure 3: consumption — Table Definition5
- Figure 4: payments — Table Definition.....5
- Figure 5: users — Table Definition.....6
- Figure 6: Foreign Key Relationships.....6
- Figure 7: Stored Procedures — Insert Examples.....7
- Figure 8: Stored Procedures — Update & Delete Examples.....8
- Figure 9: User-Defined Functions — Bill Status8
- Figure 10: User-Defined Functions — Customer & Units.....9
- Figure 11: Triggers — trg_AfterPaymentInsert & trg_InsteadOfDeleteCustomer10

Electricity Billing Management System

Overview

Electricity Bill Management System, is a SQL Server database designed for an Electricity Billing Management. It tracks customers, their electricity consumption, generated bills, payments made against those bills, and system login users. The database uses 5 tables, 3 foreign key relationships, 15 stored procedures, and 4 user-defined functions to keep everything organized and reusable.

Section 1 — Tables

- Below is a detailed explanation of each table along with its CREATE TABLE query.

1.1 customers

Figure 1 — CREATE TABLE Query:

```
CREATE TABLE [dbo].[customers] (  
    [customer_id] INT IDENTITY(1,1) NOT NULL, -- Auto-incremented primary key  
    [name]        VARCHAR(100)      NOT NULL,  
    [address]     VARCHAR(255)      NOT NULL,  
    [phone]       VARCHAR(20)       NULL,  
    PRIMARY KEY CLUSTERED ([customer_id] ASC)  
);
```

Figure 1: customers — Table Definition

- This table stores the personal details of every electricity subscriber — their name, home address, and phone number. Every other table (bills, consumption) connects back to this table.
- Each customer is given a unique customer_id automatically by the system (IDENTITY), so there is no need to enter it manually.

1.2 bills

Figure 2 — CREATE TABLE Query:

```
CREATE TABLE [dbo].[bills] (
    [bill_id] INT IDENTITY(1,1) NOT NULL, -- Auto primary key
    [customer_id] INT NOT NULL, -- FK → customers
    [total_units] INT NOT NULL,
    [amount] DECIMAL(10,2) NOT NULL,
    [bill_date] DATE NOT NULL,
    [status] VARCHAR(20) NULL, -- 'paid' / 'unpaid'
    PRIMARY KEY CLUSTERED ([bill_id] ASC),
    CHECK ([amount] >= 0),
    CHECK ([total_units] >= 0)
);
```

Figure 2: bills — Table Definition

- This table stores the electricity bills generated for each customer. It records the total units consumed in a billing period, the calculated amount due, the date the bill was generated, and whether it has been paid or is still unpaid.

1.3 consumption

Figure 3 — CREATE TABLE Query:

```
CREATE TABLE [dbo].[consumption] (
    [consumption_id] INT IDENTITY(1,1) NOT NULL, -- Auto primary key
    [customer_id] INT NOT NULL, -- FK → customers
    [units] INT NOT NULL,
    [record_date] DATE NOT NULL,
    PRIMARY KEY CLUSTERED ([consumption_id] ASC),
    CHECK ([units] > 0)
);
```

Figure 3: consumption — Table Definition

- This table records the electricity usage readings for each customer over time. Each row represents one meter reading — how many units were consumed and on which date.
- It links to the customers table via customer_id. A CHECK constraint ensures that the units value is always greater than zero, preventing invalid zero or negative readings.

1.4 payments

Figure 4 — CREATE TABLE Query:

```
CREATE TABLE [dbo].[payments] (  
    [payment_id] INT IDENTITY(1,1) NOT NULL, -- Auto primary key  
    [bill_id] INT NOT NULL, -- FK → bills  
    [amount_paid] DECIMAL(10,2) NOT NULL,  
    [payment_date] DATE NOT NULL,  
    PRIMARY KEY CLUSTERED ([payment_id] ASC),  
    CHECK ([amount_paid] > 0)  
);
```

Figure 4: payments — Table Definition

- This table keeps a record of every payment made against a bill. It stores the amount paid and the date of payment, linked directly to the bill being cleared.
- It connects to the bills table through bill_id, so every payment entry must reference a valid, existing bill. The CHECK constraint ensures the paid amount is always a positive value.

1.5 users

Figure 5 — CREATE TABLE Query:

```
CREATE TABLE [dbo].[users] (  
    [user_id] INT IDENTITY(1,1) NOT NULL, -- Auto primary key  
    [username] VARCHAR(50) NOT NULL UNIQUE,  
    [password] VARCHAR(50) NOT NULL,  
    PRIMARY KEY CLUSTERED ([user_id] ASC)  
);
```

Figure 5: users — Table Definition

- This table manages the login accounts for staff who operate the billing system. It stores a unique username and password for each system user.
- A UNIQUE constraint on the username column ensures no two users can share the same login name. This table is independent — it does not link to customers or bills.

Section 2 — Relationships

The three foreign key relationships below define how the tables connect to each other and enforce data integrity across the database.

Figure 6 — Foreign Key Constraint Queries:

```
-- bills.customer_id → customers.customer_id
ALTER TABLE [dbo].[bills]
    ADD FOREIGN KEY ([customer_id]) REFERENCES [dbo].[customers]([customer_id]);

-- consumption.customer_id → customers.customer_id
ALTER TABLE [dbo].[consumption]
    ADD FOREIGN KEY ([customer_id]) REFERENCES [dbo].[customers]([customer_id]);

-- payments.bill_id → bills.bill_id
ALTER TABLE [dbo].[payments]
    ADD FOREIGN KEY ([bill_id]) REFERENCES [dbo].[bills]([bill_id]);
```

Figure 6: Foreign Key Relationships

- customers → bills: One customer can have many bills. The bills table references customers through customer_id. You cannot create a bill for a customer who does not exist.
- customers → consumption: One customer can have many consumption records across different dates.
- bills → payments: One bill can have one or more payment entries linked to it. The payments table references bills through bill_id. You cannot record a payment for a bill that does not exist.

Section 3 — Stored Procedures

The database has 15 stored procedures that handle all Insert, Update, and Delete operations for every table. Instead of writing raw SQL each time, the application calls these procedures by name.

3.1 Insert Procedures

Figure 7 — InsertCustomer & InsertBill:

```
CREATE PROCEDURE [dbo].[InsertCustomer]
    @name    VARCHAR(100),
    @address VARCHAR(255),
    @phone   VARCHAR(20) = NULL
AS
    INSERT INTO customers (name, address, phone)
    VALUES (@name, @address, @phone);

CREATE PROCEDURE [dbo].[InsertBill]
    @customer_id INT, @total_units INT,
    @amount DECIMAL(10,2), @bill_date DATE, @status VARCHAR(20)
AS
    INSERT INTO bills (customer_id, total_units, amount, bill_date, status)
    VALUES (@customer_id, @total_units, @amount, @bill_date, @status);
```

Figure 7: Stored Procedures — Insert Examples

- Each table has a dedicated Insert procedure. For example, InsertCustomer accepts the name, address, and an optional phone number and adds a new customer row. Similarly, InsertBill adds a new bill record tied to an existing customer.

3.2 Update & Delete Procedures

Figure 8 — UpdateBill & DeleteBill:

```
CREATE PROCEDURE [dbo].[UpdateBill]
    @bill_id INT, @customer_id INT, @total_units INT,
    @amount DECIMAL(10,2), @bill_date DATE, @status VARCHAR(20)
AS
    UPDATE bills
    SET customer_id=@customer_id, total_units=@total_units,
        amount=@amount, bill_date=@bill_date, status=@status
    WHERE bill_id = @bill_id;

CREATE PROCEDURE [dbo].[DeleteBill]
    @bill_id INT
AS
    DELETE FROM bills WHERE bill_id = @bill_id;
```

Figure 8: Stored Procedures — Update & Delete Examples

- Update procedures allow modifying every field of an existing record by passing the record's ID along with the new values. For example, UpdateBill can change the status of a bill from "unpaid" to "paid".
- Delete procedures remove a record by its primary key ID. For example, DeleteBill removes a specific bill entry. Similar procedures exist for all five tables.

Section 4 — User-Defined Functions

The database includes 4 scalar functions that return single computed values. These are used for quick lookups and status checks without writing full queries each time.

4.1 Bill Status Functions

Figure 9 — fn_GetBillStatus & fn_IsBillPaid:

```
-- Returns bill status string for a given bill_id
CREATE FUNCTION [dbo].[fn_GetBillStatus] (@bill_id INT)
RETURNS VARCHAR(20) AS
BEGIN
    DECLARE @status VARCHAR(20);
    SELECT @status = status FROM bills WHERE bill_id = @bill_id;
    RETURN @status;
END;

-- Returns 'Yes' if bill is paid, 'No' otherwise
CREATE FUNCTION [dbo].[fn_IsBillPaid] (@bill_id INT)
RETURNS VARCHAR(3) AS
BEGIN
    IF EXISTS (SELECT 1 FROM bills WHERE bill_id=@bill_id AND status='paid')
        RETURN 'Yes';
    RETURN 'No';
END;
```

Figure 9: User-Defined Functions — Bill Status

- fn_GetBillStatus takes a bill_id and returns its current status string (e.g., "paid" or "unpaid"). This is useful when the application needs to display the bill status anywhere.
- fn_IsBillPaid takes a bill_id and returns a simple "Yes" or "No" answer. This makes it easy for the application to check payment status with a true/false style result.

4.2 Customer & Consumption Functions

Figure 10 — *fn_GetCustomerName* & *fn_GetTotalUnits*:

```
-- Returns customer name for a given customer_id
CREATE FUNCTION [dbo].[fn_GetCustomerName] (@customer_id INT)
RETURNS VARCHAR(100) AS
BEGIN
    DECLARE @name VARCHAR(100);
    SELECT @name = name FROM customers WHERE customer_id = @customer_id;
    RETURN @name;
END;

-- Returns total electricity units consumed by a customer
CREATE FUNCTION [dbo].[fn_GetTotalUnits] (@customer_id INT)
RETURNS INT AS
BEGIN
    DECLARE @units INT;
    SELECT @units = SUM(units) FROM consumption WHERE customer_id = @customer_id;
    RETURN ISNULL(@units, 0);
END;
```

Figure 10: User-Defined Functions — Customer & Units

- *fn_GetCustomerName* takes a *customer_id* and returns the full name of that customer. This avoids joining the customers table every time a name needs to be displayed.
- *fn_GetTotalUnits* takes a *customer_id* and returns the total units consumed across all recorded readings for that customer. If no records exist, it safely returns 0 instead of NULL.

Section 5 — Triggers

The database includes 2 triggers that enforce business rules automatically without requiring any action from the application.

5.1 Trigger Definitions

Figure 11 — Trigger Queries:

```
-- TRIGGER 1: Auto-marks a bill as 'paid' when payment covers the full amount
create trigger trg_AfterPaymentInsert
on payments
after insert
as
begin
    update bills
    set status = 'paid'
    where bill_id in (select bill_id from inserted)
        and amount <= (select isnull(sum(amount_paid), 0)
                        from payments where bill_id = bills.bill_id);
end;

insert into payments values (2, 3200.00, '2026-04-05');
select * from bills where bill_id = 2;
```

Figure 11: Triggers — trg_AfterPaymentInsert

- trg_AfterPaymentInsert is an AFTER INSERT trigger on the payments table. Every time new payment row is inserted, it automatically checks whether the total amount paid now covers bill's full amount.

Figure 12 — Trigger Queries:

```
-- TRIGGER 2: Blocks customer deletion if they have bills or consumption records
create trigger trg_InsteadOfDeleteCustomer
on customers
instead of delete
as
begin
    if exists (select 1 from consumption where customer_id in
              (select customer_id from deleted))
        or exists (select 1 from bills where customer_id in
                  (select customer_id from deleted))
    begin
        raiserror('Cannot delete customer with existing consumption or bills.', 16, 1);
    end
    else
    begin
        delete from customers where customer_id in
            (select customer_id from deleted);
    end;
end;

delete from customers where customer_id = 1;
```

Figure 11: Triggers — trg_InsteadOfDeleteCustomer

- `trg_InsteadOfDeleteCustomer` is an INSTEAD OF DELETE trigger on the customers table. Rather than deleting a customer directly, it first checks whether that customer has any linked bills or consumption records. If related data exists, it raises an error to prevent accidental data loss. Only if the customer has no linked records does it proceed with the actual deletion.

Section 5 — Summary

The flow below shows how all tables in the database connect to each other:

- customers → consumption (tracks meter readings per customer)
- customers → bills (generates billing records per customer)
- bills → payments (records payment made against each bill)
- users → (login management table)

The combination of foreign keys, CHECK constraints, stored procedures, and user-defined functions makes this project a clean, reliable, and maintainable foundation for an electricity billing application.